
Tests unitaires

Assane SECK
Ingénieur-Informaticien



Introduction

- ▶ Un test unitaire est une procédure permettant de vérifier le bon fonctionnement d'une partie précise d'un logiciel
- ▶ En POO => on teste au niveau des classes Pour chaque classe on a une classe de test

Quelques règles

- ▶ Doit être isolé : il doit être indépendant
- ▶ N'est pas un test de bout en bout : il agit que sur une portion de code
- ▶ Doit être déterministe : le résultat doit être le même pour les mêmes entrées
- ▶ Est le plus petit et simple possible
- ▶ Ne teste pas d'enchaînement d'action
- ▶ Etre lancé le plus souvent possible : intégration continue
- ▶ Etre lancé le plus tôt possible : détection des bug plus rapide
- ▶ Couvrir le plus de code possible
- ▶ Etre lancé a chaque modification

Avantages

- ▶ Garantie la non régression
- ▶ Détection de bug plus facile
- ▶ Aide à isoler les fonctions
- ▶ Aide à voir l'avancement d'un projet (TDD)

Outils de tests

PHP	JS	SQL	JAVA
PHPUnit SimpleTest ¹	JSUnit	SQLUnit	JUnit

CAS A TESTER

- ▶ Lors de l'utilisation de test unitaire on se doit de tester différents cas.
- Cas en succès : fonctionnement normal
- Cas d'erreur : test sur la gestion d'erreur
- ▶
- Cas aux limites : test de la robustesse

Quelques Cas

```
public class Operations {  
  
    public static long diviser(final long...pNombres) {  
        if(pNombres.length < 2) {  
            throw new IllegalArgumentException(  
                "Il faut au moins deux nombres en entrée");  
        }  
        long lRetour = pNombres[0];  
        for(int i=1;i<pNombres.length;i++) {  
            lRetour /= pNombres[i];  
        }  
        return lRetour;  
    }  
}
```

```
public class TestThrowable {  
  
    @Test(expected=IllegalArgumentException.class)  
    public void divisionAvecUnNombre() {  
        Operations.diviser(1);  
    }  
  
    @Test(expected=ArithmeticException.class)
```

Quelques Cas

```
public class TestDureeLimitee {  
  
    @Test(timeout=1000)  
    public void dureeRespectee() {  
  
    }  
  
    @Test(timeout=1000)  
    public void dureeNonRespectee() throws InterruptedException {  
        Thread.sleep(10000);  
    }  
}
```


Quelques Cas

```
public class TestIndisponibilite {  
  
    @Test  
    public void nonIgnore1() {  
  
    }  
  
    @Test  
    public void nonIgnore2() {  
        Assert.fail("Echec");  
    }  
  
    @Ignore  
    @Test  
    public void ignore() {  
        Assert.fail("Echec ignoré");  
    }  
}
```

JUnit

- ▶ Garantie la non régression
- ▶ JUnit (<http://junit.org/>) est une bibliothèque Java facilitant l'écriture de tests unitaires.
- ▶ Elle fournit des méthodes pour tester que certaines conditions sont satisfaites, et signaler une erreur dans le cas contraire.
- ▶ Elle fournit une infrastructure permettant de lancer tous les tests contenus dans une classe donnée, et
- ▶ Signaler les éventuelles erreurs.
- ▶ Junit sera utilisé pour illustrer les idées du test unitaire,
- ▶ Ces idées ne sont pas spécifiques à JUnit ou à Java.

Quelques Cas

```
public class TestIndisponibilite {  
  
    @Test  
    public void nonIgnore1() {  
  
    }  
  
    @Test  
    public void nonIgnore2() {  
        Assert.fail("Echec");  
    }  
  
    @Ignore  
    @Test  
    public void ignore() {  
        Assert.fail("Echec ignoré");  
    }  
  
}
```

JUnit

- ▶ Pour tester une unité à l'aide de JUnit, il convient de définir une (ou plusieurs) classe de test contenant un certain nombre de méthodes de test.
- ▶ Une telle méthode se reconnaît à l'annotation `@Test` qu'on lui attache.

`@Test`

```
public void addition() {  
    assertEquals(2, 1 + 1);  
}
```

- ▶ La méthode de test doit utiliser une ou plusieurs méthodes d'assertion fournies par JUnit (`assertEquals`, `assertTrue`, etc.), vérifiant qu'une condition est vraie.

Junit, Méthode d'assertion

- ▶ La classe Assert de JUnit fournit des méthodes statiques d'assertion, qui vérifient qu'une condition est vraie et signalent une erreur dans le cas contraire. Exemples :
- ▶ **assertTrue**(boolean b) vérifie que **b** est vrai
- ▶ **assertNull**(Object o) vérifie que **o** est nul
- ▶ **assertEquals**(Object e, Object a) vérifie que **a** est égal à **e** au moyen de la méthode equals,
- ▶ **assertEquals**(long e, long a) vérifie que **e** et **a** sont égaux
- ▶ **assertEquals**(double e, double a, double d) vérifie que la différence entre **e** et **a** est inférieure à **d**
- ▶ etc.

Junit, Méthode d'assertion

- ▶ L'ordre des arguments passés à la méthode **assertEquals** a son importance, le premier étant toujours la valeur attendue (expected value), le second la valeur obtenue effectivement (actual value).
- ▶ JUnit en tient compte pour produire ses messages d'erreur. Par exemple : etc.

```
assertEquals("apple", "orange");
```

produit le message suivant : org.junit.ComparisonFailure: expected:
but was:<orange>

Junit, Méthode d'assertion

```
public class SortTest {  
    @Test  
    public void sortsEmptyArray() {  
        int[] a1 = new int[0];  
        int[] a2 = new int[0];  
        sort(a2);  
        assertEquals(a1, a2);  
    }  
}
```

```
boolean isSorted(int[] array) {  
    for (int i = 1; i < array.length; ++i)  
        if (array[i] < array[i - 1])  
            return false;  
    return true;  
}
```

```
@Test  
public void sortsNontrivialArray() {  
    int[] a = new int[]{ 4,3,6,1,5,6,4,-1 };  
    sort(a);  
    assertTrue(isSorted(a));  
}
```

Junit, Méthode d'assertion

- ▶ L'ordre des arguments passés à la méthode **assertEquals** a son importance, le premier étant toujours la valeur attendue (expected value), le second la valeur obtenue effectivement (actual value).
- ▶ JUnit en tient compte pour produire ses messages d'erreur. Par exemple : etc.

```
assertEquals("apple", "orange");
```

produit le message suivant : org.junit.ComparisonFailure: expected:
but was:<orange>

TDD, Test Driven Development

- ▶ Le Test Driven Development (Développement Dirigé par les Tests), est une technique de développement qui impose l'écriture de tests avant même l'écriture de la première ligne de code.
- ▶ Écrire un test
- ▶ Vérifier qu'il échoue
- ▶ Écrire le code suffisant pour que le test passe
- ▶ Vérifier que le test passe
- ▶ Optimiser le code et vérifier qu'il n'y ait pas de régression

TDD, Test Driven Development

On peut découper le TDD en 5 étapes distinctes :

- ▶ Écrire un test
- ▶ Vérifier qu'il échoue
- ▶ Écrire le code suffisant pour que le test passe
- ▶ Vérifier que le test passe
- ▶ Optimiser le code et vérifier qu'il n'y ait pas de régression.

TDD, les règles

- ▶ Vous devez écrire un test qui échoue avant d'écrire votre code lui-même.
- ▶ Vous ne devez pas écrire un test plus compliqué que nécessaire.
- ▶ Vous ne devez pas écrire plus de code que nécessaire, juste assez pour faire passer le test qui échoue.

Ces 3 règles fonctionnent ensemble et font partie d'un processus qui s'appelle le cycle **rouge-vert-refactor** ("**red-green-refactor**" en anglais).

TDD, Le cycle Rouge-vert-refactor

- ▶ Le cycle rouge-vert-refactor vous guidera dans l'écriture de vos tests. Voyons d'où vient son nom.
- ▶ **Rouge** : Vous écrivez un test simple qui échouera avant même d'avoir du code qui l'accompagne. Le test échoue, évidemment, sans le code. Du coup, le test est rouge.
- ▶ **Vert** : Vous écrivez le code le plus simple possible pour faire passer le test, même si le code est un peu ridicule ou simplifié. Le test réussit, il est donc vert.
- ▶ **Refactor** : le code que vous avez écrit pour faire passer le test est peut être illogique ou trop simple. Il faut faire un refactor de ce code maintenant si possible.

```
assertEquals("bleu", couleur);
```

On écrira:

```
assertThat(couleur, is("bleu"));
```

On note ici le gain immédiat de lisibilité (surtout en anglais)...

Formatage Source Code

